

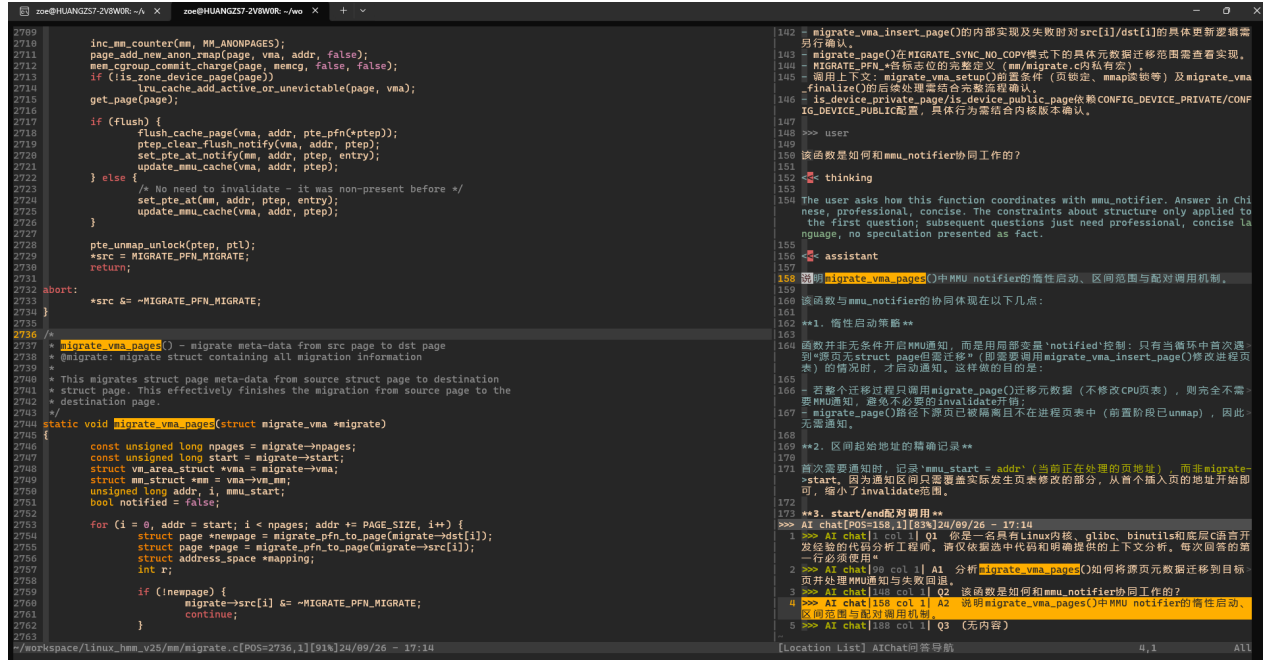
Contents

1 Vim 中使用 vim-ai 的解释与注释配置说明

1

1.1 1. vim-ai 总体功能介绍

1



1.2

1.3 2. vimrc 中实现的功能

2

1.3.1 2.1 模型接入配置

2

1.3.2 2.2 聊天 (Chat) 配置

3

1.3.3 2.3 文本编辑/补全配置

3

1.3.4 2.4 角色配置

4

1.3.5 2.5 会话打开命令 AIChatOpen

4

1.3.6 2.6 问答索引功能 AIChatIndex

4

1.3.7 2.7 键位映射与命令

9

1.3.8 2.8 其他配置

9

1.4 3. 功能总结表

9

1.5 4. vimrc 中有关 vim-ai 的完整配置项

10

1 Vim 中使用 vim-ai 的解释与注释配置说明

本文档基于 `~/.vimrc` (即 `vimrc` 文件) 中已实现的 `vim-ai` 配置进行说明, 内容包括 `vim-ai` 的总体功能、`vimrc` 中实现的具体功能与设置项、功能总结表, 以及 `vimrc` 中有关 `vim-ai` 的完整配置项。

1.1 1. vim-ai 总体功能介绍

`vim-ai` 是一个在 Vim 中通过 OpenAI 兼容接口 (本配置使用 `Lenovo glm-5.3` 模型) 调用 AI 的插件。它提供 `:AI`、`:AIEdit`、`:AIChat`、`:AICStopChat`、`:AIRedo` 等命令, 支持对当前行、可视选区或提示词进行操作, 并支持聊天、会话保存与恢复、角色配置、自定义命令和键位绑定。主要能力包括:

- **文本/代码补全:** `:AI {prompt}` 根据提示词补全, 对选区执行 `:AI` 可补全选区内容。

- **文本编辑**：:AIEdit 编辑当前行或可视选区，可附加指令，如需按角色生成注释。
- **AI 聊天**：AIChat 开始或继续对话，可带选区与角色，支持结构化问答。
- **停止生成**：AISTopChat 取消当前正在生成的 Chat 响应。
- **重试**：AIRedo 重复最近一次 AI 操作，用于重试或获取不同结果。
- **角色配置**：通过 g:vim_ai_roles_config_file 指定 .ini 角色文件，在命令中以 /role_name 引用。
- **会话保存与恢复**：.aichat 会话文件可保存并继续对话，基于 >>> user / <<< assistant 标记格式。
- **自定义与集成**：可自定义命令、键位映射，并支持 Markdown 高亮与调试开关。

插件只发送用户明确选择或写入会话的内容，不会自动索引整个工程。



1.2

1.3 2. vimrc 中实现的功能

vimrc 通过模型参数、聊天/编辑/补全配置、角色文件、自定义函数、自定义命令和键位映射，实现了以下功能。

1.3.1 2.1 模型接入配置

vimrc 中定义了公共模型参数变量 s:cfc_options，并将其复制到聊天(g:vim_ai_chat)、补全(g:vim_ai_complete)和编辑 (g:vim_ai_edit) 三者的选项中，统一接入 Lenovo glm-5.3 模型。

- **model**：模型名称，glm-5.3。
- **endpoint_url**：API 地址，以 /chat/completions 结尾。
- **auth_type**：认证方式，bearer (Bearer Token)。
- **token_file_path**：Token 文件路径，使用 expand() 展开 ~ 为家目录。
- **request_timeout**：请求超时时间 (秒)，120。
- **stream**：是否流式响应，1 启用 (SSE 分块推送)。
- **selection_boundary**：选中文本边界标记，####。
- **temperature**：采样温度，0.1 (较低，更适合代码任务)。
- **max_tokens**：单次响应最大 token 数，16384。

对应的 vimrc 配置代码如下：

```
let s:cfc_options = {
  \ 'model': 'glm-5.3',
  \ 'endpoint_url': 'https://cfc-llm-gateway.lenovo.com:4000/v1/chat/completions',
  \ 'auth_type': 'bearer',
  \ 'token_file_path': expand('~/.config/cfc-ai.token'),
  \ 'request_timeout': 120,
  \ 'stream': 1,
  \ 'selection_boundary': '####',
  \ 'temperature': 0.1,
  \ 'max_tokens': 16384,
  \ }

```

1.3.2 2.2 聊天 (Chat) 配置

`g:vim_ai_chat` 配置了聊天的服务提供方、请求参数与界面行为：

- `provider`: `openai`, 即 OpenAI 兼容接口。
- `options`: `copy(s:cfc_options)`, 复制公共模型参数。
- `ui.open_chat_command`: `vertical botright 80new`, 在最右侧垂直打开一个 80 列宽的新窗口。
- `ui.scratch_buffer_keep_open`: 0, 窗口关闭后不保留聊天 `buffer`。
- `ui.populate_options`: 0, 不在聊天头部显示生效的 `options` 配置。
- `ui.populate_all_options`: 0, 不显示全部选项。
- `ui.force_new_chat`: 0, 不强制创建新会话, 可延续历史上下文。
- `ui.paste_mode`: 1, 写入模型输出时启用 `paste` 模式, 避免自动缩进破坏格式。

对应的 `vimrc` 配置代码如下：

```
let g:vim_ai_chat = {
  \ 'provider': 'openai',
  \ 'options': copy(s:cfc_options),
  \ 'ui': {
    \ 'open_chat_command': 'vertical botright 80new',
    \ 'scratch_buffer_keep_open': 0,
    \ 'populate_options': 0,
    \ 'populate_all_options': 0,
    \ 'force_new_chat': 0,
    \ 'paste_mode': 1,
  \ },
  \ }

```

1.3.3 2.3 文本编辑/补全配置

- `g:vim_ai_edit`: 文本编辑配置, `provider` 为 `openai`, `options` 复制公共模型参数, 用于 `:AIEdit` 改写代码。
- `g:vim_ai_complete`: 代码补全配置, `provider` 为 `openai`, `options` 复制公共模型参数, 用于 `:AI` 补全。

对应的 `vimrc` 配置代码如下：

```
let g:vim_ai_edit = {
  \ 'provider': 'openai',
  \ 'options': copy(s:cfc_options),
  \ }

```

```
let g:vim_ai_complete = {
    \ 'provider': 'openai',
    \ 'options': copy(s:cfc_options),
    \ }
```

1.3.4 2.4 角色配置

vimrc 通过 `g:vim_ai_roles_config_file` 指定角色文件 `~/.config/vim-ai/roles.ini`，并在命令中以 `/role_name` 引用。其中主要角色包括：

- `[code_explain.chat]`：用于解释选中代码，固定输出结构（技术概要、执行流程、数据结构、关键函数调用、状态与并发机制、错误处理、外部依赖等），并要求以“摘要:”开头生成会话导航目录。
- `[kernel_comment.edit]`：用于为选中的内核代码添加英文注释，仅注释非显而易见的控制流、状态转换、同步、引用计数、假设和错误路径，遵循 Linux 内核注释风格 (`/* ... */`)，且不修改任何原始代码。

角色各项均设置 `options.temperature = 0.1` 以降低输出随机性。

对应的 vimrc 配置代码如下：

```
" vim-ai role 配置文件
let g:vim_ai_roles_config_file = expand('~/.config/vim-ai/roles.ini')
```

对应的 `roles.ini` (`~/.config/vim-ai/roles.ini`) 内容如下：

[code_explain.chat]

```
prompt = 你是一名具有 Linux 内核、glibc、binutils 和底层 C 语言开发经验的代码分析工程师。请仅依据选中代码和明确提供的上下文信息回答问题。
options.temperature = 0.1
```

[kernel_comment.edit]

```
prompt = Add concise English comments to the selected Linux kernel code. Comment only non-obvious control flow, state transitions, synchronization, reference counts, assumptions and error paths.
options.temperature = 0.1
```

1.3.5 2.5 会话打开命令 AIChatOpen

通过自定义命令 `AIChatOpen` 在右侧垂直分屏中以 80 列宽度打开已有 `.aichat` 会话文件：

```
command! -nargs=1 -complete=file AIChatOpen
    \ execute 'vertical rightbelow 80split ' . fnameescape(<q-args>)
```

- 支持文件路径补全 (`-complete=file`)。
- 打开后可直接在文件末尾追加问题并执行 `:AIChat` 继续会话。
- 插件通过 `filetype=aichat` 识别可继续的会话文件。

1.3.6 2.6 问答索引功能 AIChatIndex

vimrc 定义了 `AIChatBuildIndex()` 函数，为当前 `.aichat` buffer 生成问答索引，并通过 `:AIChatIndex` 命令调用，结果写入 `Location List` 以便快速跳转。

- 识别 `>>> user` 与 `<<< assistant` 标记，依次编号为 `Q1`、`Q2`…与 `A1`、`A2`…。
- 对问题取其第一个非空内容行的前 80 字符作为条目文本。
- 对回答优先检索 AI 生成的“摘要:”行作为条目文本；旧会话无摘要时取回答的第一个非空行。
- 每条记录包含 `bufnr`、`lnum`、`col` 与 `text`，并通过 `setloclist()` 写入，标题为“AIChat 问答导航”。

- 若当前 buffer 不是 aichat 文件则提示 “当前 buffer 不是 aichat 文件”；若无内容可索引则提示 “当前会话中没有可索引的问答”，并执行 lopen 打开 Location List。

对应的 vimrc 配置代码如下：

```
command! AIChatIndex call AIChatBuildIndex()

" 返回指定行之后的第一个非空内容行
function! s:AIChatFirstContent(lines, start, end) abort
    let l:index = a:start

    while l:index <= a:end
        let l:text = trim(a:lines[l:index - 1])

        if !empty(l:text)
            \ && l:text !~# '^>>>'
            \ && l:text !~# '^<<<'
            return [l:index, l:text]
        endif

        let l:index += 1
    endwhile

    return [a:start, '(无内容)']
endfunction

" 为当前 aichat buffer 生成 Location List 索引
function! AIChatBuildIndex() abort
    if &filetype !=# 'aichat'
        echohl WarningMsg
        echom 'AIChatIndex: 当前 buffer 不是 aichat 文件'
        echohl None
        return
    endif

    let l:lines = getline(1, '$')
    let l:entries = []
    let l:question_number = 0
    let l:answer_number = 0
    let l:total = len(l:lines)
    let l:index = 1

    while l:index <= l:total
        let l:line = trim(l:lines[l:index - 1])

        if l:line =~# '^>>>\s*user\s*$'
            let l:question_number += 1
```

```

let l:next_marker = l:total + 1
let l:search = l:index + 1

while l:search <= l:total
  if trim(l:lines[l:search - 1]) =~# '^(>>>\\|<<<\\)'
    let l:next_marker = l:search
    break
  endif
  let l:search += 1
endwhile

let [l:content_line, l:content] =
  \ s:AIChatFirstContent(
  \   l:lines,
  \   l:index + 1,
  \   l:next_marker - 1)

call add(l:entries, {
  \ 'bufnr': bufnr('%'),
  \ 'lnum': l:index,
  \ 'col': 1,
  \ 'text': printf(
  \   'Q%d  %s',
  \   l:question_number,
  \   strcharpart(l:content, 0, 80)),
  \ })

elseif l:line =~# '^<<<\\s*assistant\\s*$'
  let l:answer_number += 1

  let l:next_marker = l:total + 1
  let l:search = l:index + 1

  while l:search <= l:total
    if trim(l:lines[l:search - 1]) =~# '^(>>>\\|<<<\\)'
      let l:next_marker = l:search
      break
    endif
    let l:search += 1
  endwhile

  let l:summary_line = 0
  let l:summary = ''
  let l:search = l:index + 1

  " 优先检索 AI 生成的“摘要:”行
  while l:search < l:next_marker
    let l:text = trim(l:lines[l:search - 1])

```

```

        if l:text =~# '^ 摘要 [::]'
            let l:summary_line = l:search
            let l:summary = substitute(
                \ l:text,
                \ '^ 摘要 [::]\s*',
                \ '',
                \ '')
            break
        endif

        let l:search += 1
    endwhile

    " 旧会话没有摘要时，使用回答中的第一个非空行
    if empty(l:summary)
        let [l:summary_line, l:summary] =
            \ s:AChatFirstContent(
            \   l:lines,
            \   l:index + 1,
            \   l:next_marker - 1)
    endif

    call add(l:entries, {
        \ 'bufnr': bufnr('%'),
        \ 'lnum': l:summary_line > 0
        \           ? l:summary_line
        \           : l:index,
        \ 'col': 1,
        \ 'text': printf(
        \   'A%d  %s',
        \   l:answer_number,
        \   strcharpart(l:summary, 0, 80)),
        \ })

    endif

    let l:index += 1
endwhile

call setloclist(
    \ 0,
    \ [],
    \ 'r',
    \ {
    \   'title': 'AChat 问答导航',
    \   'items': l:entries,
    \ })

if empty(l:entries)

```

```

        echo 'AIChatIndex: 当前会话中没有可索引的问答'
        return
    endif

    lopen
endfunction

" 从 roles.ini 读取指定 section 中的 prompt
function! s:AIChatReadRolePrompt(section) abort
    if !exists('g:vim_ai_roles_config_file')
        return ''
    endif

    let l:path = expand(g:vim_ai_roles_config_file)

    if !filereadable(l:path)
        return ''
    endif

    let l:in_section = 0

    for l:line in readfile(l:path)
        let l:text = trim(l:line)

        if l:text =~# '^\[.*\]$'
            let l:in_section =
                \ l:text ==# '[' . a:section . ']'
            continue
        endif

        if l:in_section && l:text =~# '^prompt\s*='
            return trim(substitute(
                \ l:text,
                \ '^prompt\s*=\s*',
                \ '',
                \ ''))
        endif
    endfor

    return ''
endfunction

```

另外定义了辅助函数：

- s:AIChatFirstContent(lines, start, end): 返回指定行范围内第一个非空内容行（跳过 >>> / <<< 标记行），用于生成索引条目文本。
- s:AIChatReadRolePrompt(section): 从 g:vim_ai_roles_config_file 指定的 roles.ini 中读取指定 [section] 的 prompt 字段，供后续自定义功能调用。

1.3.7 2.7 键位映射与命令

vimrc 通过 <leader> (,) 定义了一系列与 vim-ai 相关的快捷键:

键位	模式	绑定命令	功能说明
,ae	Visual	:AIChat /code_explain	解释选中的代码
,ac	Visual	:AIEdit /kernel_comment	为选中的内核代码添加英文注释
,aw	Normal	:AIChat	打开或继续 Chat 会话
,as	Normal	:AISTopChat	停止正在生成的 AIChat 响应
,ao	Normal	:AIChatOpen	打开已有会话文件 (等待输入路径)
,at	Normal	:AIChatIndex	生成问答导航索引
,al	Normal	:lclose	关闭 Location List

对应的 vimrc 配置代码如下:

```
noremap <silent> <leader>at :AIChatIndex<CR>
" Visual 模式: 解释选中的代码
xnoremap <silent> <leader>ae :AIChat /code_explain<CR>

" Visual 模式: 为选中的内核代码添加英文注释
xnoremap <silent> <leader>ac :AIEdit /kernel_comment<CR>

nnoremap <leader>ao :AIChatOpen
" 停止正在生成的 AIChat 响应
nnoremap <silent> <leader>as :AISTopChat<CR>

nnoremap <leader>al :lclose<CR>

nnoremap <leader>aw :AIChat<CR>
```

1.3.8 2.8 其他配置

- **Markdown 高亮:** g:vim_ai_chat_markdown = 1, 启用完整的 Markdown 高亮。

对应的 vimrc 配置代码如下:

```
" enable full markdown highlighting
let g:vim_ai_chat_markdown = 1
```

1.4 3. 功能总结表

功能类别	配置项 / 命令 / 键位	用途与说明
插件安装	Plugin 'madox2/vim-ai' (Vundle 管理)	安装并启用 vim-ai
模型接入	s:cfc_options	统一模型参数
聊天配置	g:vim_ai_chat	配置 Chat 窗口打开方式、是否保留 buffer、paste 模式等
补全配置	g:vim_ai_complete	配置 :AI 补全请求参数
编辑配置	g:vim_ai_edit	配置 :AIEdit 编辑请求参数

功能类别	配置项 / 命令 / 键位	用途与说明
角色配置	g:vim_ai_roles_config_file	指定 roles.ini 角色文件路径
解释代码	,ae → :AIChat /code_explain	Visual 模式下解释选中代码
添加注释	,ac → :AIEdit /kernel_comment	Visual 模式下为选中代码添加英文注释
打开/继续会话	,aw → :AIChat	打开或继续 Chat 会话
停止生成	,as → :AISTopChat	停止正在生成的响应
打开已有会话	,ao → :AIChatOpen / :AIChatOpen <path>	右侧分屏打开 .aichat 会话文件并继续
问答索引	,at → :AIChatIndex / :AIChatBuildIndex()	为当前会话生成问答导航 Location List
关闭索引	,al → :lclose	关闭问答导航窗口
Markdown 高亮	g:vim_ai_chat_markdown = 1	启用聊天界面完整 Markdown 高亮

1.5 4. vimrc 中有关 vim-ai 的完整配置项

以下为 vimrc (~/.vimrc) 中与 vim-ai 相关的全部配置项:

```
" 通过 Vundle 插件管理安装 vim-ai
Plugin 'madox2/vim-ai'

" vim-ai configuration
"

command! -nargs=1 -complete=file AIChatOpen
    \ execute 'vertical rightbelow 80split ' . fnameescape(<q-args>)

let s:cfc_options = {
    \ 'model': 'glm-5.3',
    \ 'endpoint_url': 'https://cfc-llm-gateway.lenovo.com:4000/v1/chat/completions',
    \ 'auth_type': 'bearer',
    \ 'token_file_path': expand('~/.config/cfc-ai.token'),
    \ 'request_timeout': 120,
    \ 'stream': 1,
    \ 'selection_boundary': '#####',
    \ 'temperature': 0.1,
    \ 'max_tokens': 16384,
    \ }

let g:vim_ai_chat = {
    \ 'provider': 'openai',
    \ 'options': copy(s:cfc_options),
    \ 'ui': {
    \   'open_chat_command': 'vertical botright 80new',
    \   'scratch_buffer_keep_open': 0,
    \   'populate_options': 0,
    \   'populate_all_options': 0,
    \   'force_new_chat': 0,
    \   'paste_mode': 1,
```

```

        \ },
        \ }
let g:vim_ai_complete = {
    \ 'provider': 'openai',
    \ 'options': copy(s:cfc_options),
    \ }

let g:vim_ai_edit = {
    \ 'provider': 'openai',
    \ 'options': copy(s:cfc_options),
    \ }

" 返回指定行之后的第一个非空内容行
function! s:AiChatFirstContent(lines, start, end) abort
    let l:index = a:start

    while l:index <= a:end
        let l:text = trim(a:lines[l:index - 1])

        if !empty(l:text)
            \ && l:text !~# '^>>>'
            \ && l:text !~# '^<<<'
            return [l:index, l:text]
        endif

        let l:index += 1
    endwhile

    return [a:start, '(无内容)']
endfunction

" 为当前 aichat buffer 生成 Location List 索引
function! AiChatBuildIndex() abort
    if &filetype !=# 'aichat'
        echohl WarningMsg
        echom 'AiChatIndex: 当前 buffer 不是 aichat 文件'
        echohl None
        return
    endif

    let l:lines = getline(1, '$')
    let l:entries = []
    let l:question_number = 0
    let l:answer_number = 0
    let l:total = len(l:lines)
    let l:index = 1

```

```

while l:index <= l:total
  let l:line = trim(l:lines[l:index - 1])

  if l:line =~# '^>>>\s*user\s*$'
    let l:question_number += 1

    let l:next_marker = l:total + 1
    let l:search = l:index + 1

    while l:search <= l:total
      if trim(l:lines[l:search - 1]) =~# '^(>>>\\|<<<\\)'
        let l:next_marker = l:search
        break
      endif
      let l:search += 1
    endwhile

    let [l:content_line, l:content] =
      \ s:AIChatFirstContent(
      \   l:lines,
      \   l:index + 1,
      \   l:next_marker - 1)

    call add(l:entries, {
      \ 'bufnr': bufnr('%'),
      \ 'lnum': l:index,
      \ 'col': 1,
      \ 'text': printf(
      \   'Q%d  %s',
      \   l:question_number,
      \   strcharpart(l:content, 0, 80)),
      \ })

  elseif l:line =~# '^<<<\s*assistant\s*$'
    let l:answer_number += 1

    let l:next_marker = l:total + 1
    let l:search = l:index + 1

    while l:search <= l:total
      if trim(l:lines[l:search - 1]) =~# '^(>>>\\|<<<\\)'
        let l:next_marker = l:search
        break
      endif
      let l:search += 1
    endwhile

    let l:summary_line = 0

```

```

let l:summary = ''
let l:search = l:index + 1

" 优先检索 AI 生成的“摘要:”行
while l:search < l:next_marker
  let l:text = trim(l:lines[l:search - 1])

  if l:text =~# '^ 摘要 [: :] '
    let l:summary_line = l:search
    let l:summary = substitute(
      \ l:text,
      \ '^ 摘要 [: :]\s*',
      \ '',
      \ '')
    break
  endif

  let l:search += 1
endwhile

" 旧会话没有摘要时，使用回答中的第一个非空行
if empty(l:summary)
  let [l:summary_line, l:summary] =
    \ s:AChatFirstContent(
    \   l:lines,
    \   l:index + 1,
    \   l:next_marker - 1)
endif

call add(l:entries, {
  \ 'bufnr': bufnr('%'),
  \ 'lnum': l:summary_line > 0
  \           ? l:summary_line
  \           : l:index,
  \ 'col': 1,
  \ 'text': printf(
  \   'A%d %s',
  \   l:answer_number,
  \   strcharpart(l:summary, 0, 80)),
  \ })
endif

let l:index += 1
endwhile

call setloclist(
  \ 0,
  \ [],

```

```

        \ 'r',
        \ {
        \   'title': 'AIChat 问答导航',
        \   'items': l:entries,
        \ })
if empty(l:entries)
    echo 'AIChatIndex: 当前会话中没有可索引的问答'
    return
endif

    lopen
endfunction

" 从 roles.ini 读取指定 section 中的 prompt
function! s:AIChatReadRolePrompt(section) abort
    if !exists('g:vim_ai_roles_config_file')
        return ''
    endif

    let l:path = expand(g:vim_ai_roles_config_file)

    if !filereadable(l:path)
        return ''
    endif

    let l:in_section = 0

    for l:line in readfile(l:path)
        let l:text = trim(l:line)

        if l:text =~# '^\[.*\]$'
            let l:in_section =
                \ l:text ==# '[' . a:section . ']'
            continue
        endif

        if l:in_section && l:text =~# '^prompt\s*='
            return trim(substitute(
                \ l:text,
                \ '^prompt\s*=\s*',
                \ '',
                \ ''))
        endif
    endfor

    return ''
endfunction

```

```

" vim-ai role 配置文件
let g:vim_ai_roles_config_file = expand('~/.config/vim-ai/roles.ini')

command! AIChatIndex call AIChatBuildIndex()

noremap <silent> <leader>at :AIChatIndex<CR>
" Visual 模式: 解释选中的代码
xnoremap <silent> <leader>ae :AIChat /code_explain<CR>

" Visual 模式: 为选中的内核代码添加英文注释
xnoremap <silent> <leader>ac :AIEdit /kernel_comment<CR>

nnoremap <leader>ao :AIChatOpen
" 停止正在生成的 AIChat 响应
nnoremap <silent> <leader>as :AIStopChat<CR>

nnoremap <leader>al :lclose<CR>

nnoremap <leader>aw :AIChat<CR>

" enable full markdown highlighting
let g:vim_ai_chat_markdown = 1

说明: 以上 <leader> 定义为 , (let mapleader = ","). s:cfc_options 中的 endpoint_url、API
Key (Token 文件) 与精确模型标识均以 Lenovo 内部服务文档为准。

```